

output sequences by only controlling s_1 .³ Still, a transformer may not be able to represent a function that achieves that in practice, i.e., it is not obvious if there is a surjective map from $\{f_{s_1} : s_1 \in \mathbb{R}^{d_e}\}$ to $\{1, \dots, V\}^N$. We show that, in fact, there is a transformer f for which such a surjective map exists:

Theorem 1 (Exponential unconditional generation capacity of a single virtual token). *For any $V, N > 0$, there exists a transformer with vocabulary size V , context size N , embedding size $d_e = N$, one attention layer with two heads and a three-layer MLP such that it generates any token sequence $(Y_1, \dots, Y_N) \in \{1, \dots, V\}^N$ when conditioned on the single virtual token $s_1 = ((Y_1 - 1)/V, \dots, (Y_N - 1)/V)$.*

However, conditional generation is more interesting: given a user input $(X_1, \dots, X_{n_X})$, we want to generate a target response $(Y_1, \dots, Y_{n_Y})$. Even in the simple case of one system token, the user provides one token and the model generates one token in response ($Y_1 = f(s_1, X_1) = f_{s_1}(X_1)$), we cannot control response of the model to any user input with the system token. There are V^V maps from X_1 to Y_1 , but s_1 can take on only V values: $|\{f_{s_1} : s_1 \in 1, \dots, V\}| = V < V^V$. Hence, tokens cannot be used to specify an arbitrary map from user input to model output. However, a single virtual token can specify any of the V^V maps, i.e., there exists a transformer $f_{s_1}(X_2)$ for which there is a surjective map from $\{f_{s_1} : s_1 \in \mathbb{R}^{d_e}\}$ to $\{1, \dots, V\}^{\{1, \dots, V\}}$.

Theorem 2 (Conditional generation capacity for a single virtual token ($n_X = n_Y = 1$)). *For any $V > 0$, there exists a transformer with vocabulary size V , context size $N = 2$, embedding size $d_e = V$, one attention layer with two heads and a three-layer MLP that reproduces any map $m: [1, \dots, V] \rightarrow [1, \dots, V]$ from a user input token to a model response token when conditioned on a single virtual token $s_1 = (m(1)/V, \dots, m(V)/V)$. That is, by selecting s_1 we control the model response to any user input.*

Theorem 2 builds on Theorem 1 by showing that soft prompting is also more expressive for governing the *conditional* behavior of a transformer model. This also holds for longer responses $n_Y > 1$ by increasing the length of the soft prompt, or longer user inputs $n_X > 1$, by increasing the depth of the model. We provide proofs in Appendix A, as well as working Python implementations.

This section showed that soft prompting, and by implication, prefix-tuning, possess greater expressiveness than prompting. As we can fully determine the map from user input to model response using virtual tokens, our findings may appear to suggest that soft prompting is as powerful as full fine-tuning. However, this is not at all the case. There are structural constraints on the capabilities of soft prompting and prefix-tuning; they cannot facilitate the learning of an entirely new task. The following section elucidates this discrepancy and reconciles these seemingly contradictory results.

4 PREFIX-TUNING CAN ONLY BIAS THE OUTPUT OF AN ATTENTION HEAD

We just saw that soft prompting and prefix-tuning can fully control the conditional behavior of a transformer. However, that assumed a specific design for the network weights. Given a fixed pretrained model, as opposed to a manually crafted one, can prefix-tuning be considered equally powerful to full fine-tuning? In this section, we show that, for an arbitrary pretrained model, a prefix S cannot change the relative attention over the content X, Y and can only bias the attention block outputs in a subspace of rank n_S , the prefix length, making it less powerful than full fine-tuning.

While full fine-tuning can alter the attention pattern of an attention head, prefix-tuning cannot. Recall the attention A_{ij} position i gives to position j for a trained transformer (Equation (1)):

$$A_{ij} = \frac{\exp(T/\sqrt{k} \mathbf{x}_i^\top \mathbf{W}_Q^\top \mathbf{W}_K \mathbf{x}_j)}{\sum_{r=1}^p \exp(T/\sqrt{k} \mathbf{x}_i^\top \mathbf{W}_Q^\top \mathbf{W}_K \mathbf{x}_r)} = \frac{\exp(T/\sqrt{k} \mathbf{x}_i^\top \mathbf{H} \mathbf{x}_j)}{\sum_{r=1}^p \exp(T/\sqrt{k} \mathbf{x}_i^\top \mathbf{H} \mathbf{x}_r)}, \quad (5)$$

where $\mathbf{W}_Q^\top \mathbf{W}_K = \mathbf{H}$. Full fine-tuning can enact arbitrary changes to $\mathbf{W}_Q$ and $\mathbf{W}_K$ and hence, assuming the input does not change (e.g., at the first attention layer), we get the following attention:

$$A_{ij}^{\text{ft}} = \frac{\exp(T/\sqrt{k} \mathbf{x}_i^\top \mathbf{H} \mathbf{x}_j + T/\sqrt{k} \mathbf{x}_i^\top \Delta \mathbf{H} \mathbf{x}_j)}{\sum_{r=1}^p \exp(T/\sqrt{k} \mathbf{x}_i^\top \mathbf{H} \mathbf{x}_r + T/\sqrt{k} \mathbf{x}_i^\top \Delta \mathbf{H} \mathbf{x}_r)},$$

³For example, LLaMA-7B (Touvron et al., 2023) has 24 426 unique completions when prompted with each of its 32 000 tokens and we found a non-exhaustive set of 46 812 unique 10-token-long sequences by controlling the first virtual token. Hence, in practice, one can generate more outputs by soft prompting than by prompting.

where the changes to $\mathbf{W}_Q$ and $\mathbf{W}_K$ are folded into $\Delta\mathbf{H}$. It is clear that by varying $\Delta\mathbf{H}$ full fine-tuning can change the attention patterns arbitrarily. However, let us see how is attention affected by the presence of a prefix. For now, assume we have a prefix of length one ($\mathbf{s}_1$) at position 0.

$$\mathbf{A}_{i0}^{\text{pt}} = \frac{\exp(\frac{T}{\sqrt{k}} \mathbf{x}_i^\top \mathbf{H} \mathbf{s}_1)}{\exp(\frac{T}{\sqrt{k}} \mathbf{x}_i^\top \mathbf{H} \mathbf{s}_1) + \sum_{r=1}^p \exp(\frac{T}{\sqrt{k}} \mathbf{x}_i^\top \mathbf{H} \mathbf{x}_r)}, \quad \mathbf{A}_{ij}^{\text{pt}} = \frac{\exp(\frac{T}{\sqrt{k}} \mathbf{x}_i^\top \mathbf{H} \mathbf{x}_j)}{\exp(\frac{T}{\sqrt{k}} \mathbf{x}_i^\top \mathbf{H} \mathbf{s}_1) + \sum_{r=1}^p \exp(\frac{T}{\sqrt{k}} \mathbf{x}_i^\top \mathbf{H} \mathbf{x}_r)} \quad \text{for } j \geq 1.$$

The numerator of $\mathbf{A}_{ij}^{\text{pt}}$ is the same as in Equation (5), i.e., the prefix does not affect it. It only adds the term $\exp(\frac{T}{\sqrt{k}} \mathbf{x}_i^\top \mathbf{H} \mathbf{s}_1)$ to the denominator. Therefore, the attention position i gives to the content positions $j \geq 1$ is simply scaled down by the attention it now gives to the prefix. If *tomato* attends the most to *salad* in a particular context, no prefix can change that. This becomes evident by rewriting $\mathbf{A}_{ij}^{\text{pt}}$ as the attention of the pretrained model scaled by the attention “stolen” by the prefix:

$$\mathbf{A}_{ij}^{\text{pt}} = \mathbf{A}_{ij} \sum_{r=1}^p \mathbf{A}_{ir}^{\text{pt}} = \mathbf{A}_{ij} (1 - \mathbf{A}_{i0}^{\text{pt}}). \quad (6)$$

Hence, prefix-tuning cannot affect the relative attention patterns across the content, it will only scale them down. In other words, one cannot modify what an attention head attends to via prefix-tuning.⁴

Prefix-tuning only adds a bias to the attention block output. Let us see how this attention scaling down affects the output of the attention block. Following Equation (2), the output at position i for the pretrained ($\mathbf{t}_i$), the fully fine-tuned ($\mathbf{t}_i^{\text{ft}}$) and the prefix-tuned ($\mathbf{t}_i^{\text{pt}}$) models are as follows:⁵

$$\begin{aligned} \mathbf{t}_i &= \sum_{j=1}^p \mathbf{A}_{ij} \mathbf{W}_V \mathbf{x}_j, & \mathbf{t}_i^{\text{ft}} &= \sum_{j=1}^p \mathbf{A}_{ij}^{\text{ft}} (\mathbf{W}_V + \Delta\mathbf{W}_V) \mathbf{x}_j, \\ \mathbf{t}_i^{\text{pt}} &= \mathbf{A}_{i0}^{\text{pt}} \mathbf{W}_V \mathbf{s}_1 + \sum_{j=1}^p \mathbf{A}_{ij}^{\text{pt}} \mathbf{W}_V \mathbf{x}_j \stackrel{(6)}{=} \mathbf{A}_{i0}^{\text{pt}} \mathbf{W}_V \mathbf{s}_1 + \sum_{j=1}^p \mathbf{A}_{ij} (1 - \mathbf{A}_{i0}^{\text{pt}}) \mathbf{W}_V \mathbf{x}_j = \mathbf{A}_{i0}^{\text{pt}} \mathbf{W}_V \mathbf{s}_1 + (1 - \mathbf{A}_{i0}^{\text{pt}}) \mathbf{t}_i. \end{aligned} \quad (7)$$

Hence, prefix-tuning only biases the attention block value at each position i towards the constant vector $\mathbf{W}_V \mathbf{s}_1$, which is independent of the content ($\mathbf{x}_1, \dots, \mathbf{x}_p$). I.e., the prefix-tuned activation is a linear combination of the pretrained activation and the constant vector $\mathbf{W}_V \mathbf{s}_1$. The content only affects the scale $\mathbf{A}_{i0}^{\text{pt}}$ of the bias via the amount of attention on the prefix. In contrast, in full fine-tuning $\Delta\mathbf{W}_Q$, $\Delta\mathbf{W}_K$ and $\Delta\mathbf{W}_V$ allow for a content-dependent change of the attention and value computation. These results hold for suffix-tuning (placing the prefix after the input) but *not* for suffix soft-prompting. We validate that this indeed is the case when prefix-tuning real-world transformers. In Figures 5 and 6, we show that a prefix applied to LLaMA’s first layer does not change the relative attention distribution over the content positions X and results in a bias with a constant direction.

Longer prefixes define larger subspaces for the bias but are not fully utilized in practice. In the case of a longer prefix ($\mathbf{s}_1, \dots, \mathbf{s}_{n_S}$), the bias vector is in a subspace of dimensionality n_S : $\mathbf{t}_i^{\text{pt}} = \sum_{j=1}^{n_S} \mathbf{A}_{i,S_j}^{\text{pt}} \mathbf{W}_V \mathbf{s}_j + (1 - \sum_{j=1}^{n_S} \mathbf{A}_{i,S_j}^{\text{pt}}) \mathbf{t}_i$, where i goes over the content and j over the prefix positions. Larger prefixes thus have a larger subspace to modify the attention block output. The specific direction is determined by the relative distribution of attention across the prefix positions. However, when we examine the distribution of attention across the prefix positions for various inputs as in Appendix B, it appears that the prefixes do not span this subspace. Regardless of the input, the attention $\mathbf{A}_{i,S_j}^{\text{pt}}$ over the prefix positions remains nearly constant. Thus, prefix-tuning does not seem to make full use of the space that the vectors $\mathbf{W}_V \mathbf{s}_j$ span. We hypothesise that this is due to the two competing optimization goals for the vectors $\mathbf{s}_j$: at the same time they need to “grab attention” when interacting with $\mathbf{W}_K$ and determine the bias direction when multiplied with $\mathbf{W}_V$.

So, is prefix-tuning equivalent to full fine-tuning or is it less powerful than full fine-tuning? In Section 3, we showed that prefix-tuning, in principle, has a large capacity to influence the behavior of the model. But then, in this section, we showed that it has some severe limitations, including not being able to affect the attention pattern and only biasing the attention layer activations. These two results seem to be contradicting one another, so how do we reconcile them?

The constructions for the results in Section 3 (described in Appendix A) are simply an algorithm that extracts the completion from a lookup table encoded in the virtual tokens. The attention patterns

⁴Likhoshesterov et al. (2021) show that a fixed attention head can approximate any sparse attention pattern. However, they require control over all the input embeddings while we can only control the prefix ones.

⁵He et al. (2021a) show a similar analysis but do not study the expressiveness of prefix-tuning.

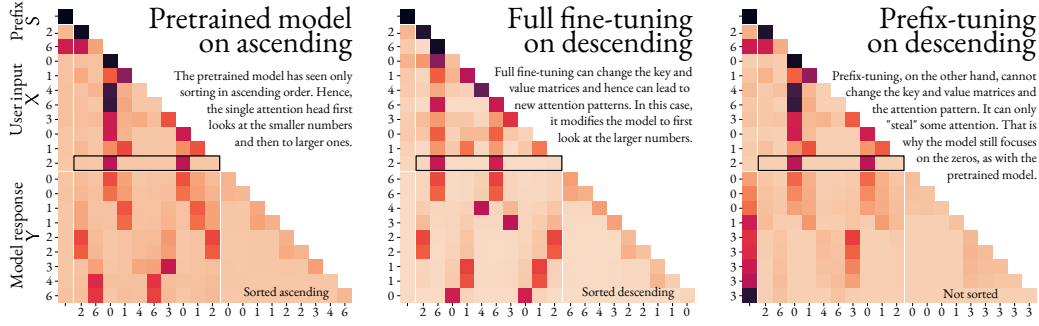


Figure 1: Attention patterns of a small transformer pretrained on sorting in ascending order. The model is given the prefix S and user input X and generates Y autoregressively. We have highlighted the attention when the first response Y_1 is being generated. Full fine-tuning sorts in descending order but prefix-tuning cannot as it cannot update the learned attention. Note how the relative attention of X to X in the left and right plots is exactly the same: the prefix cannot change the attention pattern for the same inputs. The relative attention of X to X in the center plot is very different because full fine-tuning can arbitrarily change W_Q and W_K .

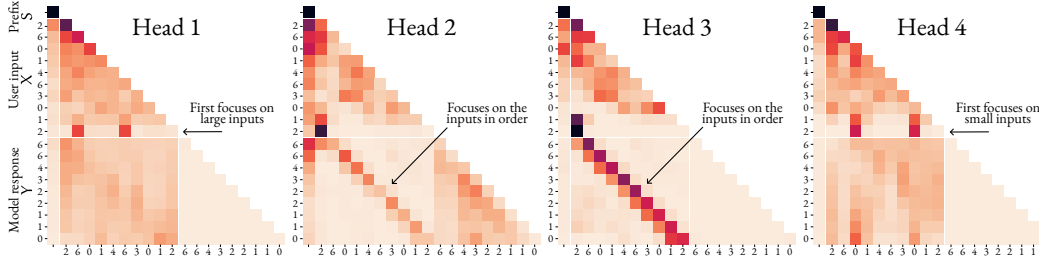


Figure 2: Model pretrained on the four tasks. The four attention heads specialize in the skills necessary to solve these tasks: look at the elements in order, look first at the smallest elements or first at the largest elements.

are simply extracting the current position embedding and the virtual token and hence the attention does not depend on the actual content in the tokens. There is no need to learn a new attention pattern to learn a different map from input to output.⁶ Furthermore, the virtual token designates the map precisely by acting as a bias. Therefore, the observations in these two sections do not contradict one another. Soft prompting and prefix-tuning can be on par with full fine-tuning but only in very limited circumstances: when all the knowledge is represented in the virtual token as a lookup table and the model simply extracts the relevant entry. Transformers do not behave like this in practice. Models are typically trained with token inputs rather than virtual tokens. Moreover, if we had a lookup table of the responses to each input we would not need a learning algorithm in the first place.

Therefore, the limitations from this section hold for real-world pretrained transformers. Then how come prefix-tuning has been reported to achieve high accuracy and often to be competitive to full fine-tuning? The next section aims to explain when and why prefix-tuning can work in practice.

5 THE BIAS CAN ELICIT SKILLS FROM THE PRETRAINED MODEL

Pretraining potentially exposes a model to different types of completions for the same token sequence. For a string like *I didn't enjoy the movie*, the model may have seen completions such as *I found the acting to be sub par*, *This is negative sentiment* or *Je n'ai pas aimé le film*. Hence, a pretrained model could do text completion, sentiment analysis, or translation. Still, the input does not fully determine the desired completion type and the model can generate any one of them. Hence, following our results from Section 4, we hypothesise that prefix-tuning cannot gain *new knowledge*

⁶In a follow-up work (Petrov et al., 2024), we utilize this observation to show that, in fact, there exist pretrained weights for which a transformer can be a universal approximator for sequence-to-sequence functions when prefixed. This is not in contradiction with the present results as these transformers can approximate any function without having to modify their attention mechanism.